MXFP8/NVFP4低精度模拟与反量化（CUDA）

1. 项目概述
2. MXFP8 格式与实现
   2.1 E4M3 数据 格式
   2.2 一级缩放策略 E8M0 Block Scale
   2.3 打包布局
   2.4、量化基础实现思路
   2.5 反量化基础实现思路
3. NVFP4 格式与实现
   3.1 E2M1 数据格式
   3.2 两级缩放策略 FP8 E4M3 Block Scale + FP32 Global Scale
   3.3 打包布局
   3.4 量化基础思路
   3.5 反量化基础思路
4. 测试、性能评估与分析
   4.1. 测试环境与测试方法
   4.2. 误差指标、压缩率和性能指标
      4.2.1. 误差指标
         4.2.1.1. 最大绝对误差
         4.2.1.2. MAE
         4.2.1.3. MSE
      4.2.2. 压缩率
      4.2.3. 性能指标
   4.3. MXFP8 NCU性能分析
      4.3.1. MXFP8 Quantize
         4.3.1.1. MXFP8 Quantize Baseline
            4.3.1.1.1. V1 NCU
         4.3.1.2. MXFP8 Quantize：提高每 Warp 工作量
            4.3.1.2.1. V2：1 warp → 2 MX blocks

4.3.1.2.2. V3：1 warp → 4 MX blocks

4.3.1.2.3. V4：1 warp → 8 MX blocks

4.3.1.3. MXFP8 Quantize：优化 Scale Store

4.3.1.3.1. V4-Packed

4.3.1.4. MXFP8 Quantize：Scale Calculation SIMT 优化

4.3.1.4.1. V5

4.3.1.5. MXFP8 Quantize 最终结果

4.3.2. MXFP8 Dequantize

4.3.2.1. MXFP8 Dequantize Baseline

4.3.2.1.1. V1 NCU

4.3.2.2. MXFP8 Dequantize：提高每 Warp 工作量

4.3.3. MXFP8 最终总结

4.4. NVFP4 NCU性能分析

4.4.1. NVFP4 Pipeline 优化

4.4.2. NVFP4 Global Amax Kernel

4.4.2.1. V1：256 blocks

4.4.2.2. V1： 256 → 864 blocks

4.4.2.3. V2/V3 提升不大

4.4.3. NVFP4 Quantize Kernel

4.4.3.1. NCU V1 Baseline

4.4.3.2. V2：Vectorized Input Load

4.4.3.3. V3：Branch-Reduced E2M1 Encoding

4.4.3.4. V4：共享倒数计算

4.4.3.5. V5：1 Thread → 4 Values

4.4.3.6. V6：1 Thread → 8 Values （继续线程粗化）

4.4.3.7. NVFP4 Quantize 最终结果

4.4.4. NVFP4 Dequantize Kernel

4.4.4.1. V1

4.4.4.2. V2：Vectorized Output Store

4.4.4.3. V3：Bit-Level E2M1 Decode

4.4.4.4. V4：Bit-Level E4M3 Scale Decode

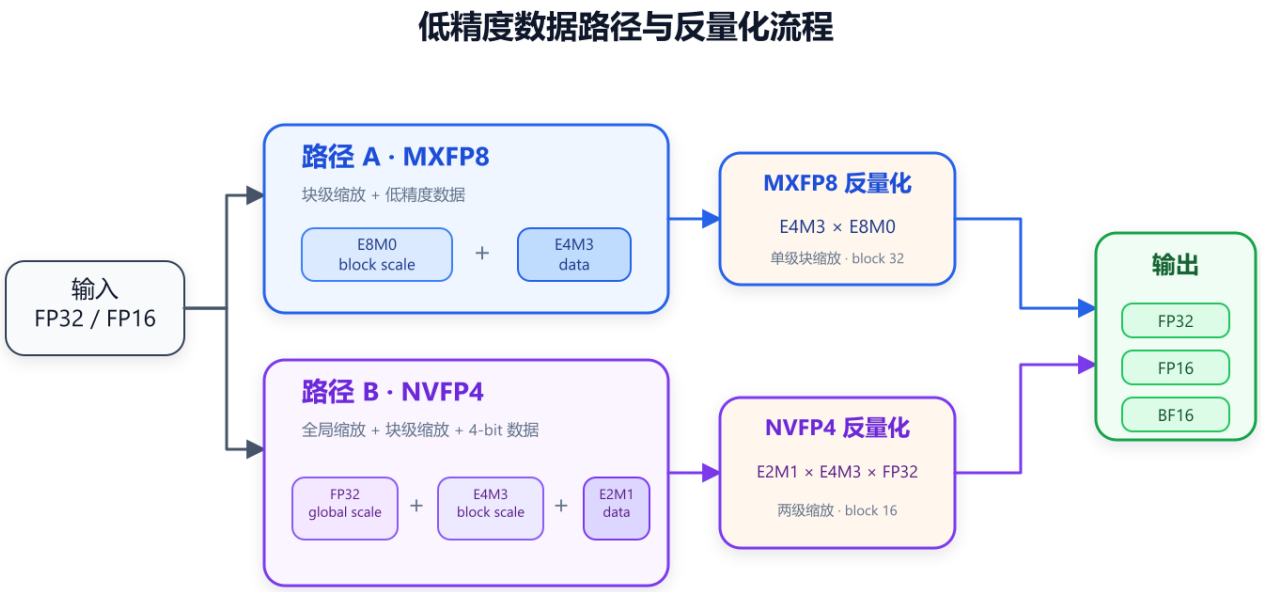
4.4.4.5. NVFP4 最终结果

5. 输出展示

1. 项目概述

本项目实现了 MXFP8 与 NVFP4 两种低精度数据格式的 CUDA 软件模拟，包括从 FP32 / FP16 输入进行低精度量化、scale 计算、低精度数据打包，以及从低精度表示反量化到 FP32 / FP16 / BF16。

整个实现使用软件模拟，不依赖 GPU 原生 FP8 / FP4 Tensor Core，也不依赖 Transformer Engine、cuDNN 或 CUTLASS 提供的低精度量化接口。因此实现能够明确展示低精度格式的编码规则、缩放策略、bit-level packing（位级打包）以及反量化过程。



2. MXFP8 格式与实现

MXFP8采用 E4M3 低精度数据格式，配合每 32 个元素共享的 **E8M0 Block Scale。**

2.1 E4M3 数据 格式

MX规范定义了两种 FP8 元素：

- E4M3
- E5M2

MXFP8 可以使用其中任何一种，本项目支持 MXFP8 E4M3。

E4M3 浮点格式

8-bit 布局



特殊值与数值范围

项目	E4M3 编码与数值
指数偏置	7
规格化数公式	$x = (-1)^S \cdot 2^{E-7} \cdot (1 + \frac{M}{2^3})$
次正规数公式	$x = (-1)^S \cdot 2^{-6} \cdot \frac{M}{2^3}$
可表达幅值	2^{-9} ～ 448（离散浮点值；正负对称）
Infinity	N/A（不支持）
NaN	S 1111 111_2
零	S 0000 000_2（支持 +0 和 −0）
最大正规数	S 1111 110_2 = $\pm 2^8 \times 1.75 = \pm 448$
最小正规数	S 0001 000_2 = $\pm 2^{-6}$
最大次正规数	S 0000 111_2 = $\pm 2^{-6} \times 0.875$
最小次正规数	S 0000 001_2 = $\pm 2^{-9}$

2.2 一级缩放策略 E8M0 Block Scale

每 32 个元素共享一个基于局部最大值计算的 E8M0 块级 scale，再将元素量化为 FP8。

E8M0 浮点格式

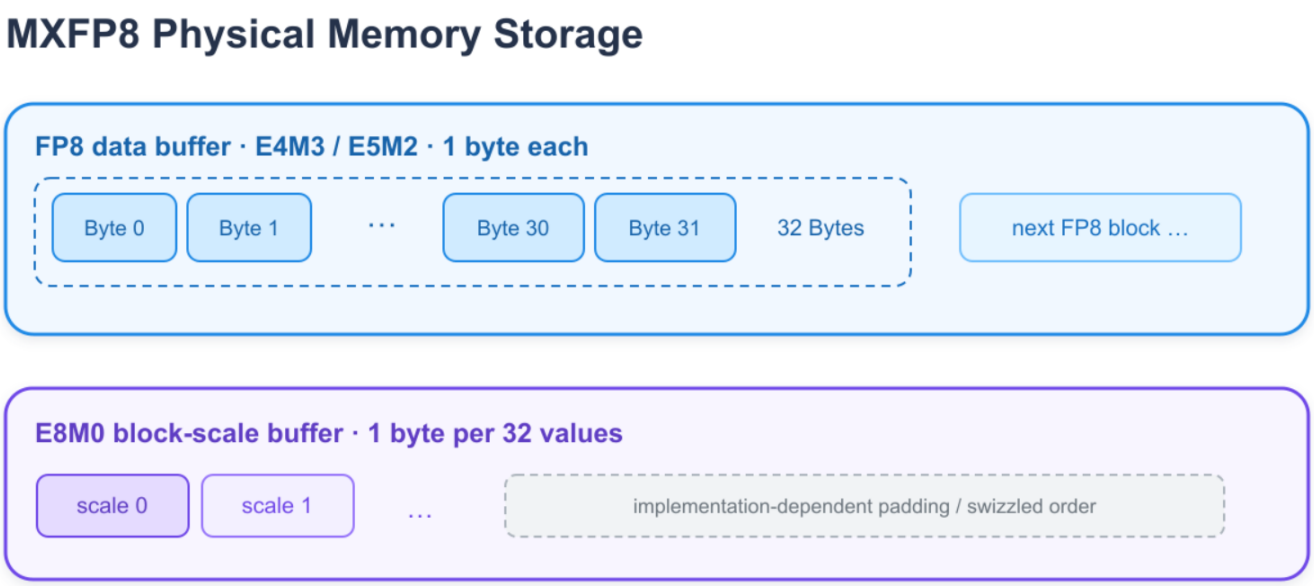
8-bit 布局



特殊值与数值范围

项目	E8M0 编码与数值
指数偏置	**127**
数值公式	**X = 2^(E−127)**
指数范围	**−127 ~ 127**
Infinity	**N/A（不支持）**
NaN	**11111111_2**
零	**N/A（不支持）**
最小值	**$00000000_2 = 2^{-127}$**
最大值	**$11111110_2 = 2^{127}$**
共享尺度规则	**若尺度为 NaN，整个 32 元素块均视为 NaN**

2.3 打包布局



2.4、量化基础实现思路

1. 接收 FP32 或 FP16 输入，以及元素数量 `n`。
2. 如果 `n <= 0`，直接返回；否则以 256 个线程为一个 CUDA block 启动量化 kernel。
3. 每个线程处理一个输入元素，并根据线程编号计算：
   - 当前元素的全局索引 `tid`
   - 在线程束中的位置 `lane`
   - 所属的 32 元素 MXFP8 数据块 `mx_block`
4. FP16 输入先转换为 FP32（使用 CUDA 内置函数）。 FP16 的所有有限数，包括次正规数，都可以被 FP32 精确表示，所以这个 FP16→FP32 转换本身不会产生舍入误差。 对于超出输入范围的尾部线程，使用 0 参与后续归约。
5. 每个 warp 对 32 个元素的绝对值执行最大值归约，得到块内最大绝对值 `amax`。
6. warp 的第 0 个线程根据 `amax` 计算共享缩放指数 `k`。缩放因子为：

$$scale = 2^k$$

   `k` 取能够满足以下条件的最小整数：

$$\frac{amax}{2^k} \le 448$$

   其中 448 是 E4M3 能表示的最大有限值。

   特殊情况为：

   - `amax <= 0` 或全为 NaN 时，令 `k = 0`
   - `amax` 为无穷大时，令 `k = 127`

- 最终将 `k` 限制在 `[-127, 127]`

7. 将缩放指数编码成 E8M0：

$$scale_code = k + 127$$

并保存到当前 MXFP8 数据块对应的 `scales[mx_block]` 中。

8. warp 的第 0 个线程通过 `__shfl_sync` 将 `k` 广播给其余线程。
9. 每个有效线程将自己的输入元素编码为 E4M3。概念上相当于先计算：

$$y = \frac{x}{2^k}$$

实际实现不会真正执行 FP32 除法，而是直接根据 `k` 调整 FP32 指数。

10. E4M3 编码过程如下：
    - 读取 FP32 的符号位、指数位和尾数位。
    - `+0` 编码为 `0x00`，`-0` 编码为 `0x80`。
    - 正 NaN 编码为 `0x7F`，负 NaN 编码为 `0xFF`。
    - E4M3 没有无穷大，因此正负无穷分别饱和为 `+448` 和 `-448`，对应 `0x7E` 和 `0xFE`。
    - 如果输入是 FP32 次正规数，先将其规格化为 `1.xxx × 2^E`。
    - 根据调整后的指数判断结果属于 E4M3 正规数还是次正规数。
11. 对于 E4M3 正规数：
    - 保留 3 个 fraction bit。
    - 使用 RNE（round-to-nearest-even，最接近且偶数优先）进行舍入。
    - 如果舍入导致尾数进位，则指数加一。
    - 如果结果超过 E4M3 的表示范围，则饱和到 `±448`。
12. 对于 E4M3 次正规数：
    - 按照 2^{-9} 的步长进行量化。
    - 使用 RNE 进行舍入。
    - 舍入结果为 0 时，输出带符号的零。
    - 舍入结果位于 1~7 时，输出 E4M3 次正规数。
    - 舍入结果大于或等于 8 时，进位到最小 E4M3 正规数 2^{-6}。
13. 最终，每个元素得到一个 8-bit E4M3 编码；每 32 个元素额外保存一个共享的 8-bit E8M0 缩放编码。

2.5 反量化基础实现思路

反量化相比量化要简单很多，低精度转为高精度不会和之前一样出现很多舍入情形。

1. 接收 E4M3 量化数据、E8M0 缩放数据、输出地址和元素数量 `n` 。
2. 如果 `n <= 0` ，直接返回；否则启动反量化 kernel。
3. 每个线程处理一个量化元素。如果 `tid >= n` ，该线程立即返回。
4. 根据元素索引计算其所属的 MXFP8 数据块：

$$mx_block = tid/32$$

5. 从 `scales[mx_block]` 读取该数据块共享的 E8M0 缩放编码。
6. 如果缩放编码为 `0xFF` ，将输出设置为 NaN 并返回。该值作为特殊或无效缩放编码处理。

   否则将 E8M0 编码还原成缩放指数：

$$k = scale_code - 127$$

7. 将当前元素的 E4M3 编码解码为 FP32：
   - 分离符号位、4-bit 指数和 3-bit fraction。
   - 指数和 fraction 都为 0 时，解码为带符号的零。
   - 指数为 0、fraction 非零时，按 E4M3 次正规数解码。
   - `exp = 15` 且 `frac = 7` 时，解码为 NaN。
   - 其他情况按照正规数公式计算：

$$decoded = \left(1 + \frac{fraction}{8}\right) \times 2^{exponent-7}$$

   最后恢复符号。

8. 使用共享缩放因子恢复数值：

$$restored = decoded \times 2^{k}$$

9. 将恢复后的 FP32 数值转换为目标输出格式：
   - FP32 输出直接保存。
   - FP16 输出使用 CUDA 内置函数 舍入转换。
   - BF16 输出使用 RNE 舍入转换。

3. NVFP4 格式与实现

NVFP4 继续沿用了 block scaling 的思想，但由于数据进一步从 8 bit 压缩到了 **4 bit**，仅依靠类似 MXFP8 的简单 block scale 已经很难维持足够精度 。

因此 NVFP4 做了三个重要改变：

- 数据格式从 E4M3 进一步变为 **E2M1**；
- block size 从 32 缩小到 **16**；
- scale 从单一 E8M0 scale 变成 **E4M3 Block Scale + FP32 Global Scale 两级缩放**。

3.1 E2M1 数据格式

E2M1 数据格式（NVFP4）

4-bit 布局



特殊值与数值范围

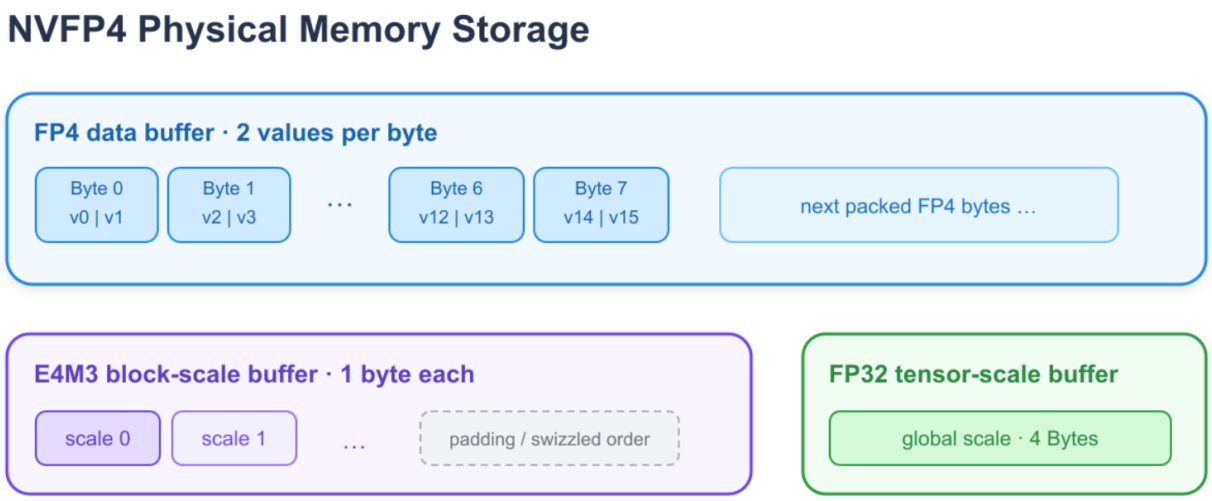
项目	E2M1 编码与数值
指数偏置	1
规格化数公式	$(-1)^s \times 2^{(E-1)} \times (1 + M/2)$
可表示幅值	0、0.5、1、1.5、2、3、4、6（正负对称）
Infinity	N/A（不支持）
NaN	N/A（不支持）
零	S 00 0_2（支持 +0 和 −0）
最小正次正规数	0 00 1_2 = 0.5
最小正规数	0 01 0_2 = 1
最大正规数	0 11 1_2 = 6；1 11 1_2 = −6

3.2 两级缩放策略 FP8 E4M3 Block Scale + FP32 Global Scale

第一级是全局缩放：先根据整个张量的最大值，计算一个所有数据共享的 FP32 scale，把整体动态范围调整到合适区间。

第二级是块缩放：再把数据每 16 个元素分成一组，每组根据自己的局部最大值计算一个 FP8 E4M3 scale，让每个小块都能更充分地利用 FP4 E2M1 的有限表示范围。

3.3 打包布局



3.4 量化基础思路

NVFP4 的低精度模拟采用三级表示：

- 整个张量共享一个 FP32 `global_scale`
- 每 16 个元素共享一个 E4M3 `block_scale`
- 每个元素使用 4-bit E2M1 编码

两个 E2M1 数值被打包进一个字节。

1. 接收 FP32 或 FP16 输入、元素数量 `n_padded` 、量化输出、block scale 输出以及 global scale 输出。

2. 如果输入非法或 `n_padded <= 0`，直接返回错误；否则先计算整个输入张量的最大有限绝对值 `global_amax`。
3. 每个线程以 grid-stride loop 遍历输入，并计算局部最大绝对值。NaN 和 Inf 不参与 `amax` 计算，只统计有限值。
4. 每个 CUDA block 在 shared memory 中执行最大值归约，最终通过 `atomicMax` 得到整个输入张量的最大值 `global_amax` 。
5. 单个线程根据 `global_amax` 计算整个张量共享的 FP32 global scale：

$$\text{global_scale} = \frac{\text{global_amax}}{448 \times 6} = \frac{\text{global_amax}}{2688}$$

   其中 448 是 E4M3 最大有限值，6 是 E2M1 最大有限值。

6. 每个线程处理两个连续输入元素， 每 8 个线程共同处理 16 个元素，对应一个 NVFP4 block。一个 warp 可以同时处理 4 个 16-element block。
7. 每个线程计算自己两个元素的最大有限绝对值，然后 8 个线程通过线程束洗牌函数得到当前 16-element block 的最大值 `block_amax` 。[NaN 和 Inf 同样不参与 `block_amax` 计算。]
8. subgroup 的第 0 个线程根据 `block_amax` 和 `global_scale` 计算理想 block scale。E4M3 block scale 使用向上舍入，从而保证 block 内最大元素经过缩放后不会超过 E2M1 的最大有限值6。

$$\text{block_scale} = Q^{\uparrow}_{\text{E4M3}}\left(\frac{\text{block_amax}}{6 \cdot \text{global_scale}}\right)$$

9. subgroup 的第 0 个线程将 E4M3 block scale 保存到 `block_scales[nv_block_id]`，并通过 `__shfl_sync` 将实际解码后的 block scale 广播给其余线程。
10. 每个有限输入元素执行两级缩放：

$$\text{scaled_value}_i = \frac{x_i}{\text{global_scale} \cdot \text{block_scale}}$$

    然后将 `scaled_value_i` 编码为 E2M1：

$$q_i = Q_{\text{E2M1}}\left(\text{scaled_value}_i\right)$$

11. E2M1 编码过程如下：

- 读取 FP32 的符号位、指数位和尾数位。
- `+0` 、 `-0` 保留符号。
- FP32 次正规数先规格化为 `1.xxx × 2^E` 。
- 正规数保留 1 个 fraction bit，并使用 RNE 舍入。
- 舍入导致尾数进位时，指数加一。

- 超出表示范围时饱和到 `±6` 。
- 次正规区域使用 RNE，只能量化到 `0` 、 `±0.5` ，或者进位到 `±1` 。

12. E2M1 没有 NaN 和 Inf 编码，因此当前实现采用饱和策略

- `+Inf` → `+6`
- `-Inf` → `-6`
- 正号 NaN → `+6`
- 负号 NaN → `-6`

13. 每个线程最终得到两个 4-bit E2M1 编码，并打包到一个字节中：

    $$(q_1 \ll 4) \mid q_0$$

    其中 `q0` 位于低 4 bit， `q1` 位于高 4 bit。

14. 最终，每 16 个元素保存：

- 16 个 4-bit E2M1 数值
- 1 个 8-bit E4M3 block scale

  整个输入张量额外共享一个 FP32 global scale。

3.5 反量化基础思路

同样的反量化不需要重新计算 `amax` 和 scale，只需要读取量化数据以及已经保存的两级缩放因子。简单许多。

1. 接收打包后的 E2M1 数据、E4M3 block scale、FP32 global scale、输出地址和元素数量 `n_padded` 。
2. 每个线程读取一个 packed byte，并负责恢复两个元素。
3. 根据线程索引计算所属的 NVFP4 block， 每 8 个线程对应 16 个输入元素 。
4. 从 packed byte 中拆出两个 E2M1 编码：

   $$q_0 = \text{packed} \ \& \ 0x0F$$

   $$q_1 = (\text{packed} \gg 4) \ \& \ 0x0F$$

5. 将两个 E2M1 编码解码为 FP32 。 对于每个 E2M1 编码，先拆出符号位、指数位和尾数位：

   $$\text{sign} = (q \gg 3) \ \& \ 0x1$$

   $$\text{exp} = (q \gg 1) \ \& \ 0x3$$

$$\text{frac} = q \;\&\; 0x1$$

当 `exp = 0` 时，为 0 或次正规数：

$$\text{value} = (-1)^{\text{sign}} \cdot \frac{\text{frac}}{2}$$

当 `exp > 0` 时，为正规数：

$$\text{value} = (-1)^{\text{sign}} \left(1 + \frac{\text{frac}}{2}\right) 2^{\text{exp}-1}$$

6. 从 `block_scales[nv_block]` 中读取当前 16-element block 的 E4M3 scale，并解码为 FP32。 先拆出符号位、指数位和尾数位：

$$\text{sign} = (\text{scale_bits} \gg 7) \;\&\; 0x1$$

$$\text{exp} = (\text{scale_bits} \gg 3) \;\&\; 0xF$$

$$\text{frac} = \text{scale_bits} \;\&\; 0x7$$

当 `exp = 0` 时，为 0 或次正规数

$$\text{block_scale} = (-1)^{\text{sign}} \cdot \frac{\text{frac}}{8} \cdot 2^{-6}$$

当 `exp > 0` 时，为正规数

$$\text{block_scale} = (-1)^{\text{sign}} \left(1 + \frac{\text{frac}}{8}\right) 2^{\text{exp}-7}$$

`exp = 15` 且 `frac = 7` 时解码为 NaN

7. 读取整个张量共享的 FP32 global scale， 使用两级缩放恢复原始数值 ：

$$\hat{x}_i = D_{\text{E2M1}}(q_i) \cdot \text{block_scale} \cdot \text{global_scale}$$

8. 将恢复后的 FP32 数值转换为目标输出格式：

- FP32 输出直接保存。
- FP16 输出使用 RNE 舍入转换。(使用cuda内置函数)
- BF16 输出使用 RNE 舍入转换。

4. 测试、性能评估与分析

4.1. 测试环境与测试方法

测试平台：NVIDIA A100-PCIE-40GB，Compute Capability 8.0

核心性能测试规模：`4096 × 4096 = 16,777,216` elements

主要工具：CUDA + Nsight Compute

分别测试三类数据分布：均匀随机矩阵，均匀随机矩阵，含异常值矩阵。

第一类是**均匀随机矩阵**，每个元素独立从U(−8,8)中采样；第二类是**正态分布矩阵**，每个元素独立采样X~N(0,3^2)； 第三类是**含异常值矩阵**，它不是简单地直接生成大范围随机数，而是先生成一个正常的小幅值背景X~N(0,1)，然后每隔4096个元素，人为插入一个大幅值 outlier（异常值）。

每种分布均测试六种输入/输出组合：

- FP32 -> Low Precision -> FP32
- FP32 -> Low Precision -> FP16
- FP32 -> Low Precision -> BF16

- FP16 -> Low Precision -> FP32
- FP16 -> Low Precision -> FP16
- FP16 -> Low Precision -> BF16

4.2. 误差指标、压缩率和性能指标

为了评价 MXFP8 与 NVFP4 量化后的数值精度，本项目对反量化结果与实际送入量化 kernel 的输入值进行比较。

4.2.1. 误差指标

本项目主要统计以下三个误差指标。

4.2.1.1. 最大绝对误差

$$\boxed{E_{\max} = \max_i |x_i - \hat{x}_i|}$$

最大绝对误差反映所有元素中最严重的一次量化偏差，对于含异常值矩阵尤其敏感。

例如一个 tensor 中绝大多数元素误差很小，但某个大幅值元素量化误差很大，那么 max_abs_error 会直接反映这一情况。

4.2.1.2. MAE

MAE（Mean Absolute Error，平均绝对误差）定义为：

$$\boxed{MAE = \frac{1}{N}\sum_{i=1}^{N}|x_i - \hat{x}_i|}$$

与最大绝对误差相比，MAE 更能反映整个 tensor 的平均量化误差水平，不容易被少量异常点完全主导。

4.2.1.3. MSE

MSE（Mean Squared Error，均方误差）定义为：

$$\boxed{MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2}$$

由于误差被平方，大误差在 MSE 中拥有更大的权重，因此 MSE 对 outlier（异常值）以及严重量化失真更加敏感。

4.2.2. 压缩率

本项目中的压缩率仅统计低精度 tensor payload（有效载荷）。

定义：

$$\boxed{Compression\ Ratio = \frac{\text{原始数据字节数}}{\text{低精度数据及 scale 字节数}}}$$

4.2.3. 性能指标

性能测试使用 CUDA Event 进行 GPU 端计时。

为了减小首次 kernel launch、cache 状态以及运行波动带来的影响，每个测试采用：

> warmup iterations = 10
> benchmark iterations = 100

最终 kernel 时间取 100 次执行的平均值。

主要统计以下性能指标。

1. 量化 Kernel 时间
2. 反量化 Kernel 时间
3. 有效内存带宽

4.3. MXFP8 NCU性能分析

固定性能比较基准：Uniform 随机输入，FP32 输入，FP32 反量化输出 。

整体优化思路为：使用 NCU profiling， 定位具体瓶颈后，进行单变量修改，然后 重新 profile ，看结果保留或回退。

4.3.1. MXFP8 Quantize

4.3.1.1. MXFP8 Quantize Baseline

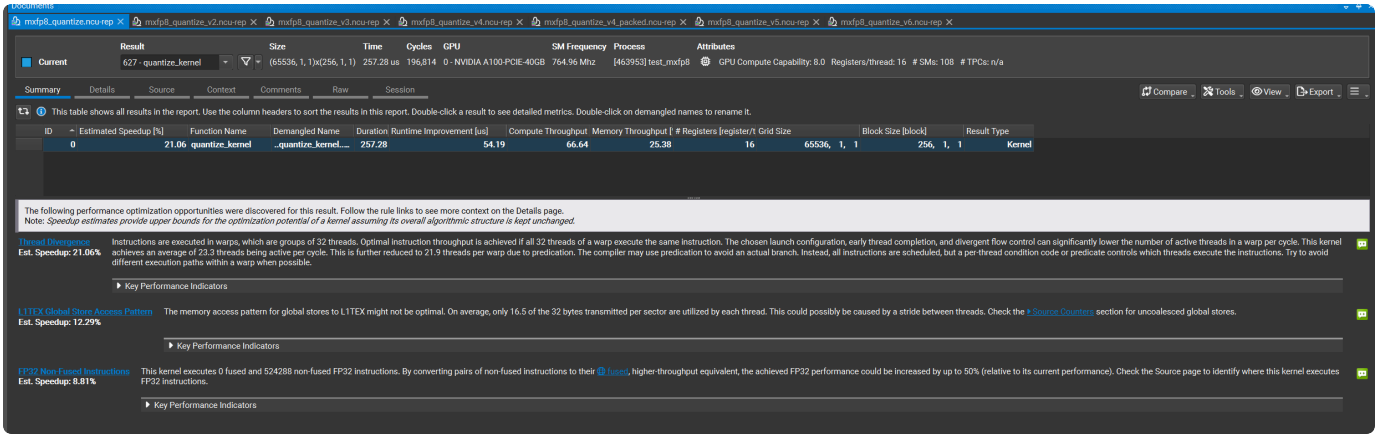
原始 Quantize 采用：

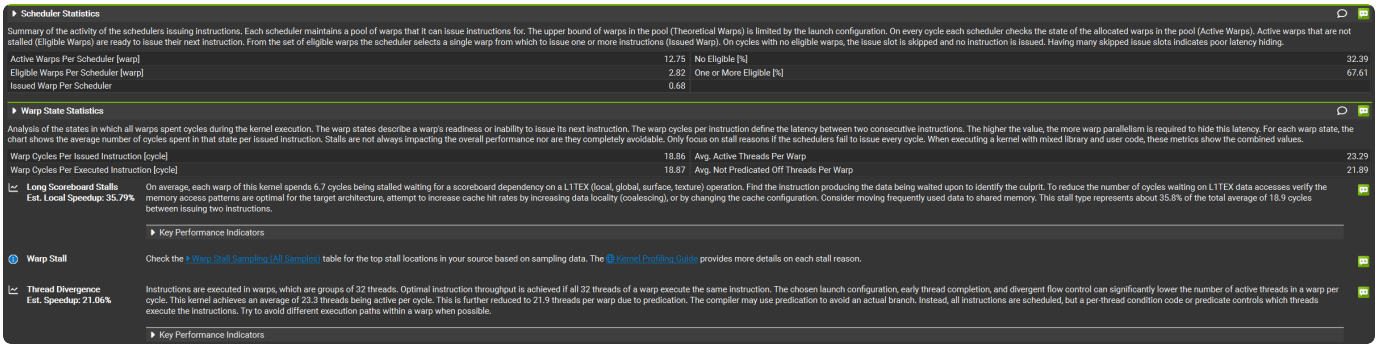
```
1 warp
→ 1 个 MX block
→ 32 values
```

一个 lane 处理一个输入元素，一个 warp 内通过 reduction 得到：

```
block_amax
    ↓
E8M0 scale
    ↓
E4M3 quantization
```

4.3.1.1.1. V1 NCU





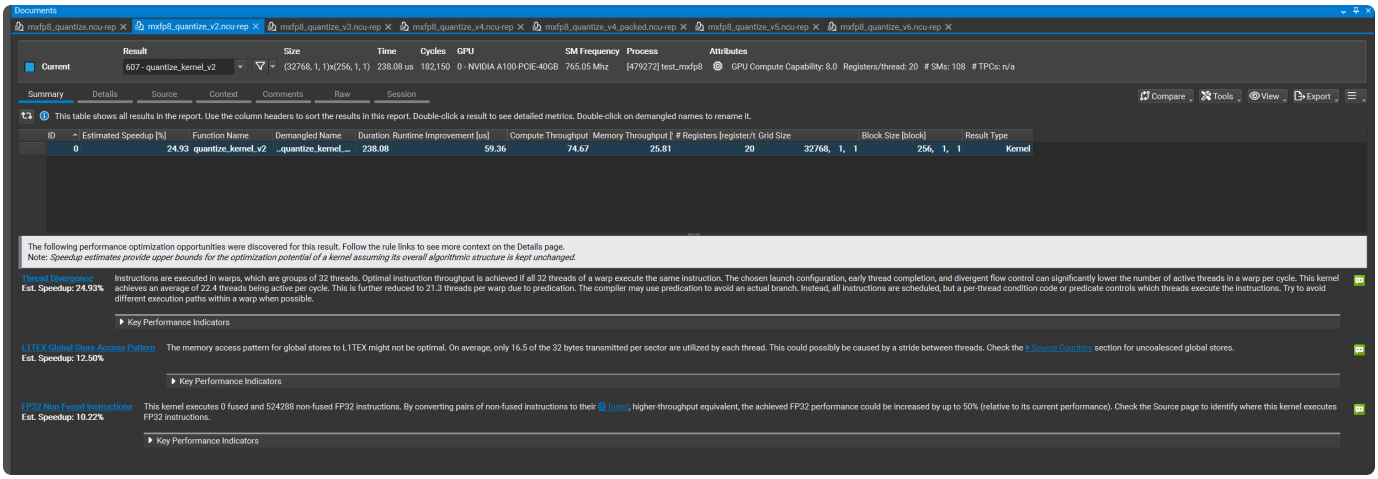
Metric	V1
Duration	**257.28 µs**
Registers / thread	16
Active Warps / Scheduler	12.75
Eligible Warps / Scheduler	2.82
Issued Warps / Scheduler	0.68
No Eligible	32.39%
Warp Cycles / Issued Inst.	18.86
Long Scoreboard	~35.8%

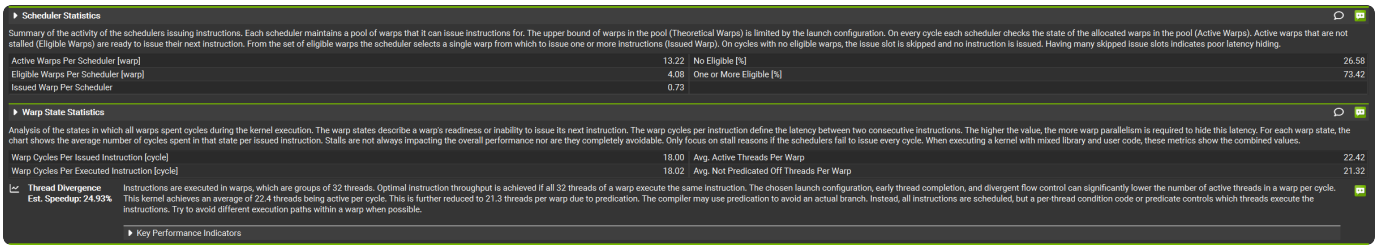
说明 kernel 存在明显：Load latency + ILP 不足

4.3.1.2. MXFP8 Quantize：提高每 Warp 工作量

因此 MXFP8 Quantize 的第一条优化主线是增加每个 warp 的工作量。最开始一个 warp 只处理一个 MX block，之后依次提高到两个、四个和八个 MX block。这样一个线程会同时维护多个彼此独立的输入值，产生更多独立 load 和计算链，从而提高 ILP 和 MLP。

4.3.1.2.1. V2：1 warp → 2 MX blocks





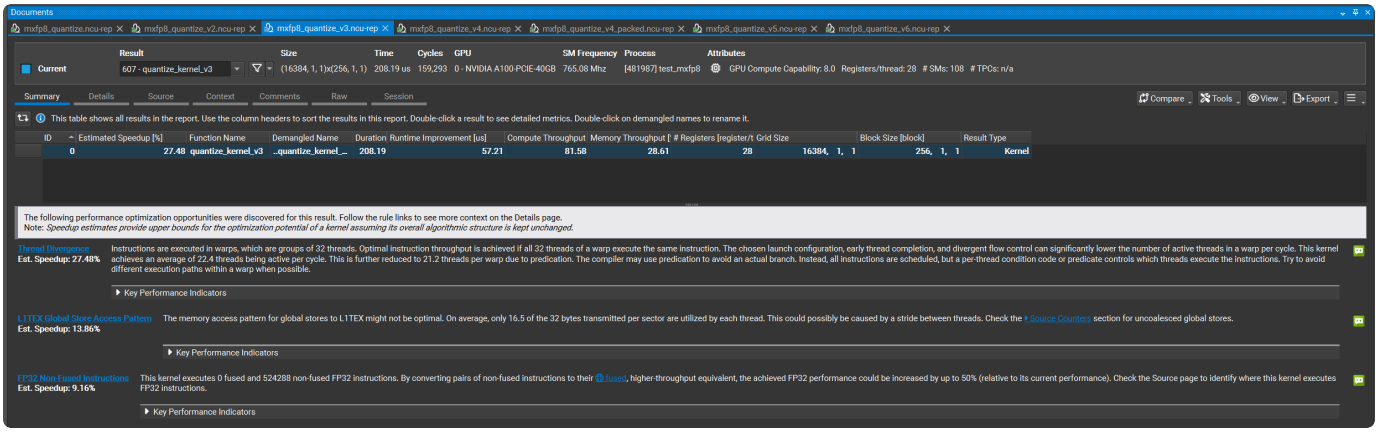
结果：

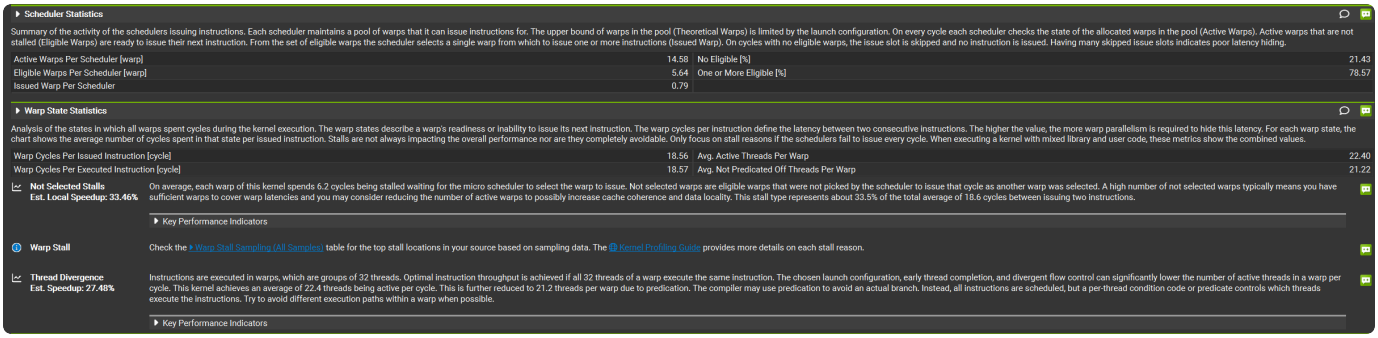
Metric	V1	V2
Duration	257.28	**238.08 μs**
Registers	16	20
Eligible Warp	2.82	**4.08**
No Eligible	32.39%	**26.58%**

性能提升：7.46%

4.3.1.2.2. V3：1 warp → 4 MX blocks

继续增加ILP



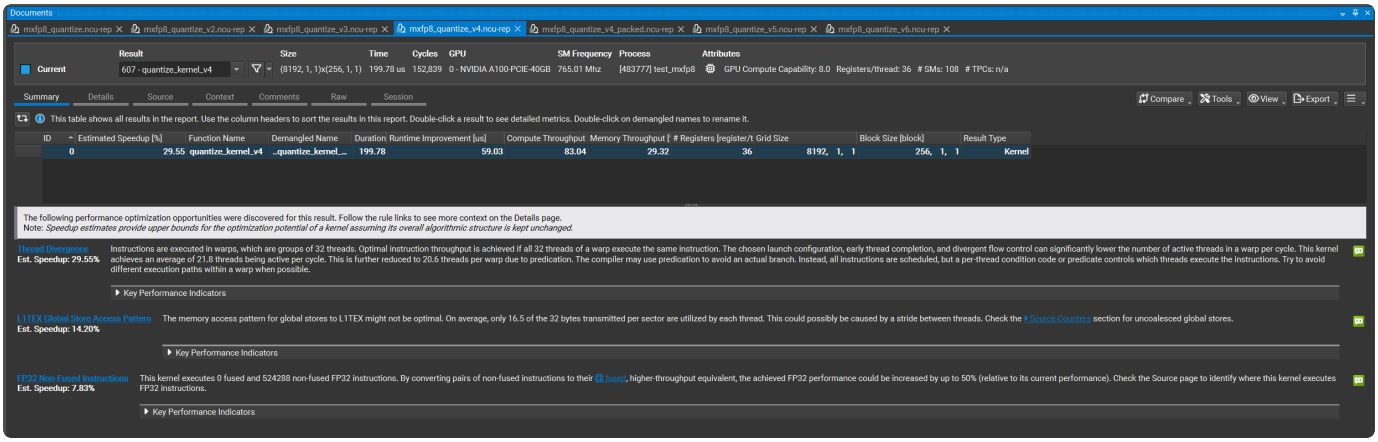


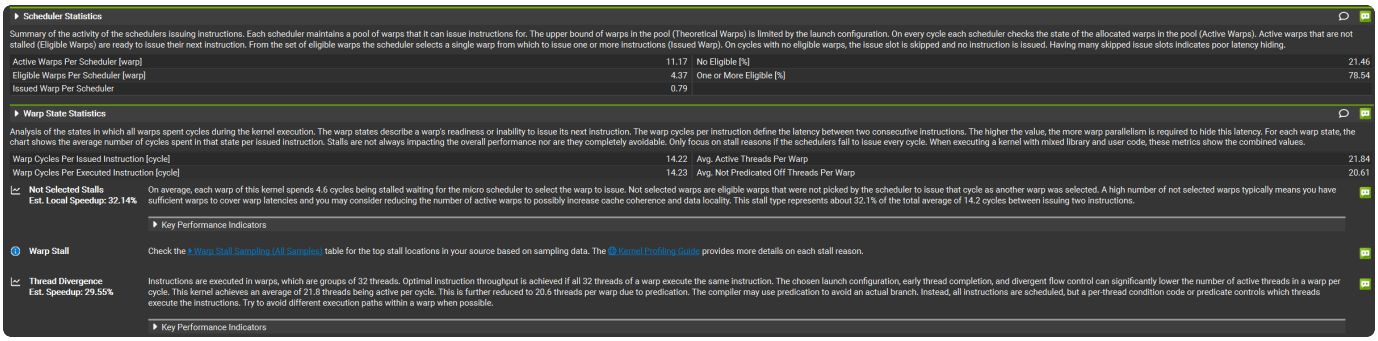
结果：

Metric	V2	V3
Duration	238.08	**208.19 μs**
Registers	20	28
Active Warp	13.22	14.58
Eligible Warp	4.08	**5.64**
No Eligible	26.58%	**21.43%**

V2 → V3：12.55%

4.3.1.2.3. V4：1 warp → 8 MX blocks





结果：

Metric	V3	V4
Duration	208.19	**199.78 μs**
Registers	28	**36**
Active Warp	14.58	**11.17**
Eligible Warp	5.64	4.37
No Eligible	21.43%	21.46%

性能仍提高约：4.04%

但：当一个 warp 处理八个 block 后，寄存器占用开始明显上升，Active Warp 数量反而有所下降。这说明继续增加单线程工作量会进入典型的 ILP、Register Pressure 和 TLP 之间的权衡区间，因此没有继续简单地扩大到十六个 block，而是开始寻找其他瓶颈。

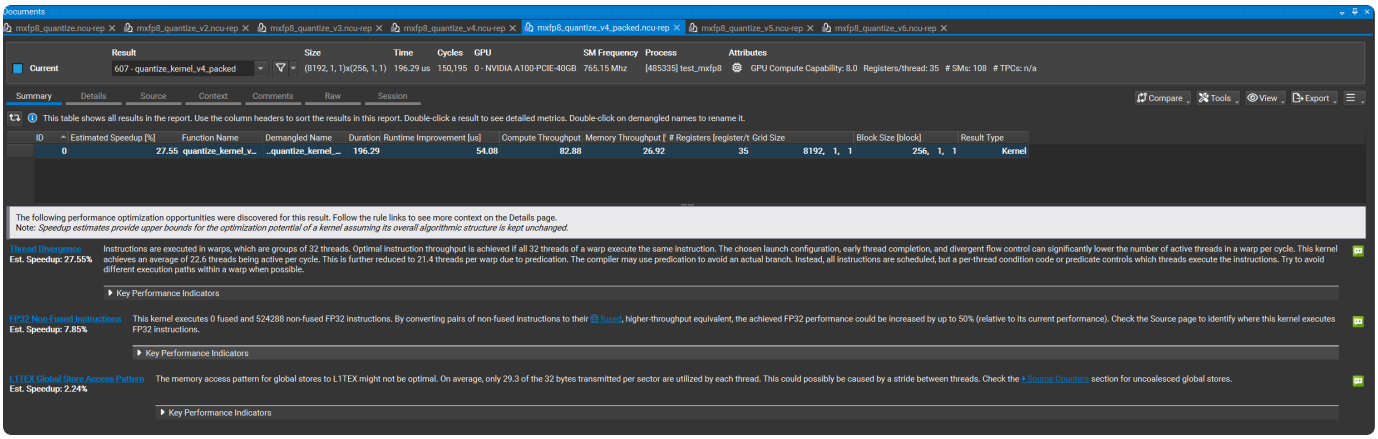
4.3.1.3. MXFP8 Quantize：优化 Scale Store

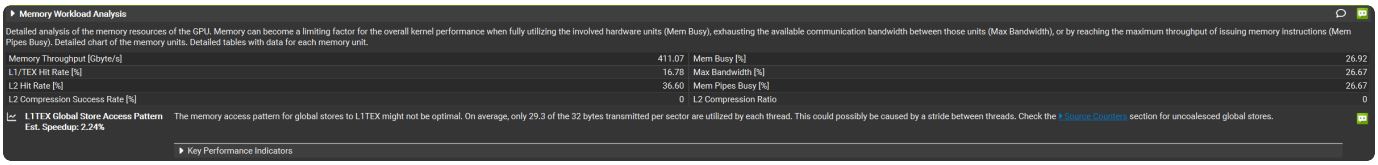
NCU 发现 global store sector utilization： 16.5 / 32 B 。

这是因为量化数据本身一个 warp 正好连续写 32 字节，但是 scale 只有 lane0 写一个字节，因此从 sector useful bytes 的角度看，平均只有 16.5/32B 被有效使用。

4.3.1.3.1. V4–Packed

由于一个 warp 此时已经一次生成八个 scale，于是将八次独立的 1–byte store 合并为一次 8–byte store。这样 sector 利用率由 16.5B 提升到了约 29.3B，与理论计算完全吻合。





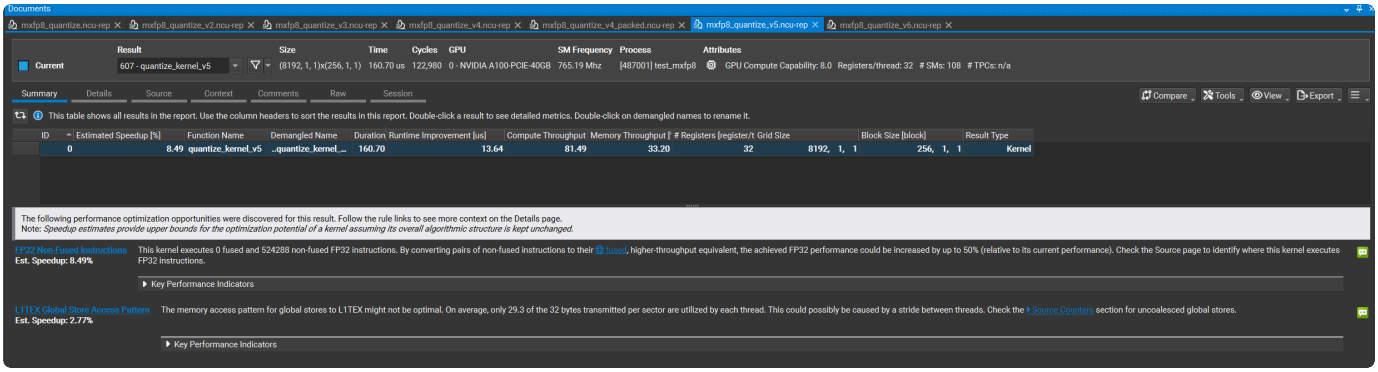
不过实际性能只提升约 1.75%，说明这个问题虽然真实存在，但不是最大的性能瓶颈。

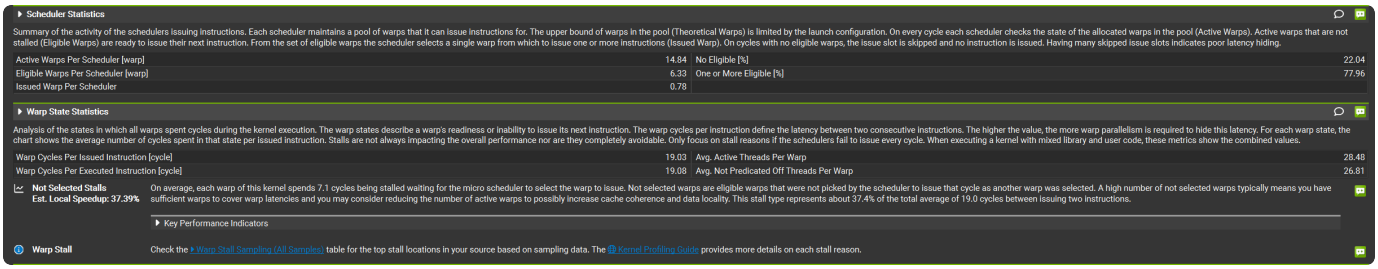
4.3.1.4. MXFP8 Quantize：Scale Calculation SIMT 优化

接下来发现更重要的问题是 scale 的计算方式。此前一个 warp 虽然处理八个 MX block，但八个 block 的 scale 都由 lane0 串行计算，这会导致大量 lane 在这段时间处于 inactive 状态。

4.3.1.4.1. V5

将 reduction 改成 butterfly all–reduce，让更多 lane 都获得 block amax，再由 lane0 到 lane7 分别负责一个 block 的 scale 计算。





结果：

Metric	V4–Packed	V5
Duration	196.29	**160.70 µs**
Registers	35	**32**
Active Warp	~11	**14.84**
Eligible Warp	~4.4	**6.33**
Avg Active Threads	—	**28.48 / 32**
Avg Predicated–On	—	**26.81 / 32**

V4–Packed → V5：18.13%

4.3.1.5. MXFP8 Quantize 最终结果

Version	核心优化	Duration
V1	Baseline	257.28 µs
V2	2 blocks/warp	238.08 µs
V3	4 blocks/warp	208.19 µs

V4	8 blocks/warp	199.78 μs
V4–Packed	packed scale store	196.29 μs
V5	distributed scale compute	**160.70 μs**

最终：

$$\text{Speedup} = \frac{257.28}{160.70} \approx 1.60\times$$

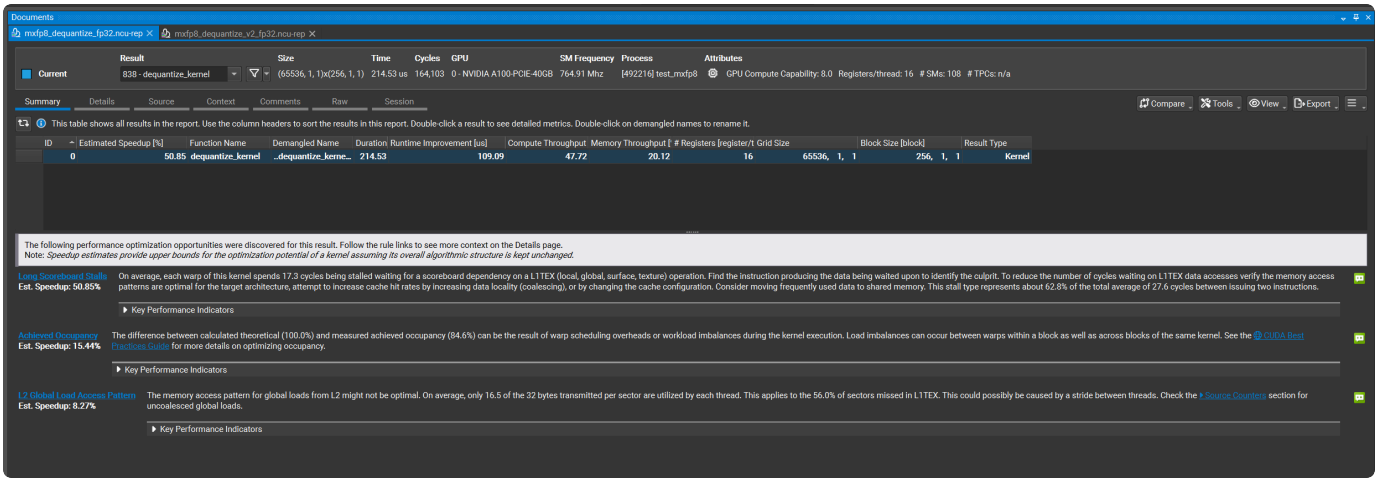
延迟下降：37.5%

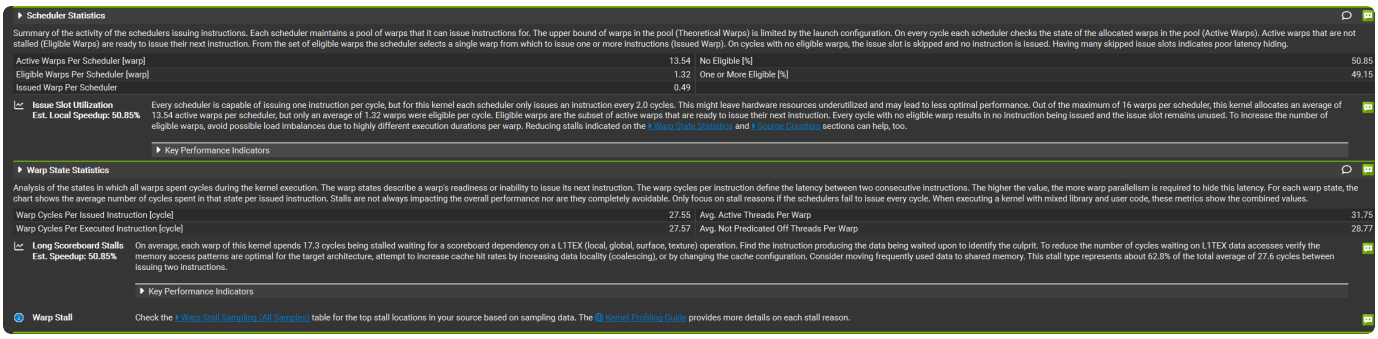
4.3.2. MXFP8 Dequantize

4.3.2.1. MXFP8 Dequantize Baseline

baseline：1 thread → 1 E4M3 → load E8M0 scale → decode → ldexp → output

4.3.2.1.1. V1 NCU





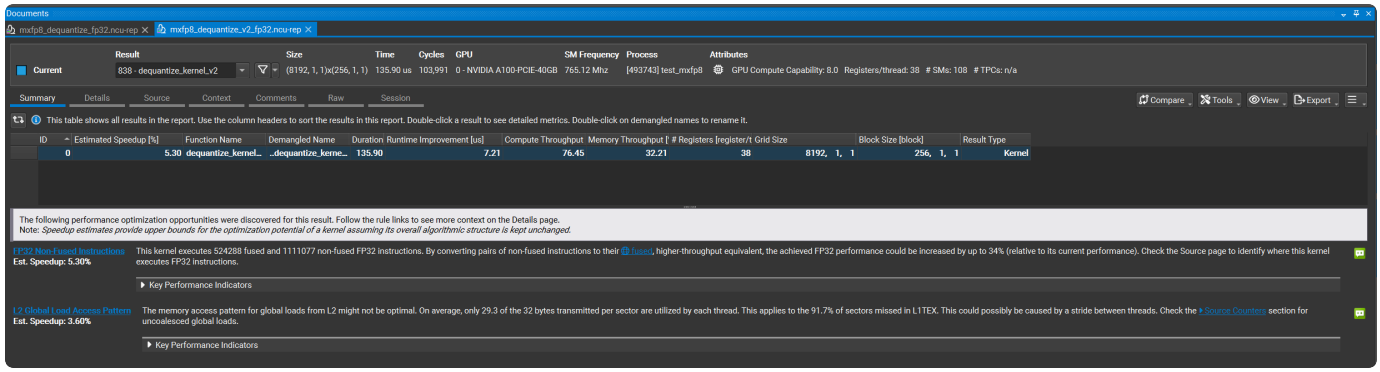
结果

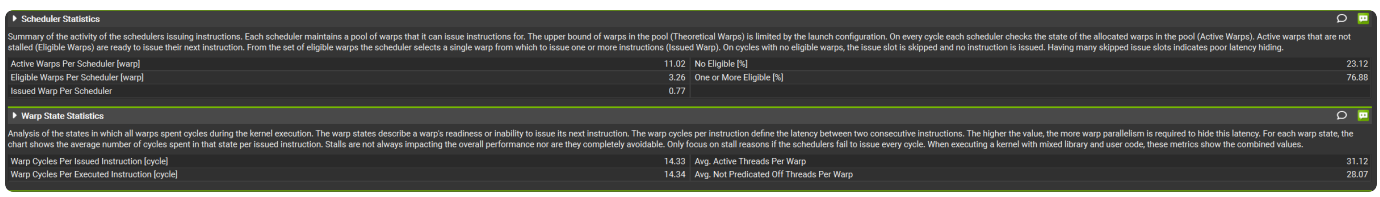
Metric	V1
Duration	**214.53 µs**
Registers	16
Active Warp	13.54
Eligible Warp	**1.32**
Issued Warp	0.49
No Eligible	**50.85%**
Warp Cycles / Issued Inst.	27.55
Long Scoreboard	**62.8%**

Baseline 中 Active Warp 很多，但 Eligible Warp 很少，No Eligible 高达约一半，Long Scoreboard 也非常突出。这说明 resident warp 虽然足够，但大量 warp 都在等待 load dependency，无法真正提供 latency hiding。

4.3.2.2. MXFP8 Dequantize：提高每 Warp 工作量

采用一个 warp 同时处理八个 MX block。每个线程同时处理八个量化值，形成多条独立 decode chain；同时由 lane0 到 lane7 连续读取八个 scale，再通过 shuffle 广播给整个 warp。这样既增加了 ILP 和 MLP，也改善了 scale load 的 sector 利用率。





结果：

Metric	V1	V2
Duration	214.53	**135.90 µs**
Registers	16	38
Active Warp	13.54	11.02
Eligible Warp	1.32	**3.26**
Issued Warp	0.49	**0.77**
No Eligible	50.85%	**23.12%**
Warp Cycles / Issued Inst.	27.55	**14.33**
Load useful bytes	16.5	**29.3 / 32B**

优化之后 Eligible Warp 从 1.32 提高到 3.26，No Eligible 从 50.85% 降到 23.12%，Warp Cycles Per Issued Instruction 也接近减半。最终 Dequantize 从 214.53 微秒降低到 135.90 微秒，约为 1.58 倍加速。

4.3.3. MXFP8 最终总结

Kernel	Baseline	Final	Speedup
Quantize	257.28 µs	**160.70 µs**	**1.60×**
Dequantize	214.53 µs	**135.90 µs**	**1.58×**

4.4. NVFP4 NCU性能分析

固定性能比较基准：Uniform 随机输入，FP32 输入，FP32 反量化输出 。

整体优化思路为：使用 NCU profiling ， 定位具体瓶颈后，进行单变量修改，然后 重新 profile ，看结果保留或回退。

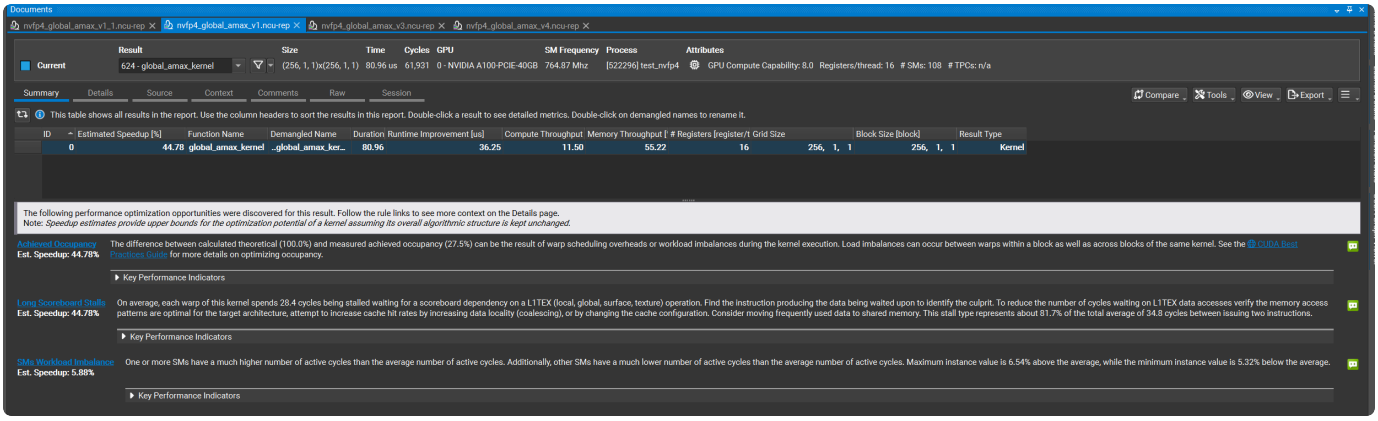
4.4.1. NVFP4 Pipeline 优化

NVFP4 Quantize 相比 MXFP8 更复杂， 因为它采用两级 scaling。首先需要遍历整个 tensor 得到 global amax，并根据它计算一个 FP32 global scale；之后每 16 个值再计算一个 E4M3 block scale，最后将数据量化成 E2M1 并进行 packing。因此 NVFP4 的优化需要同时考虑 global reduction、block-scale 计算、E2M1 转换以及整体 pipeline 开销。

4.4.2. NVFP4 Global Amax Kernel

4.4.2.1. V1：256 blocks

Baseline 为了减少 atomicMax 次数，将 grid 强行限制为 256 个 block。对于 A100 的 108 个 SM，这导致 GPU 上的 CTA 数明显不足。



NCU 显示理论 occupancy 可以达到 100%，但实际 achieved occupancy 只有 27.5%，Long Scoreboard 也很高。

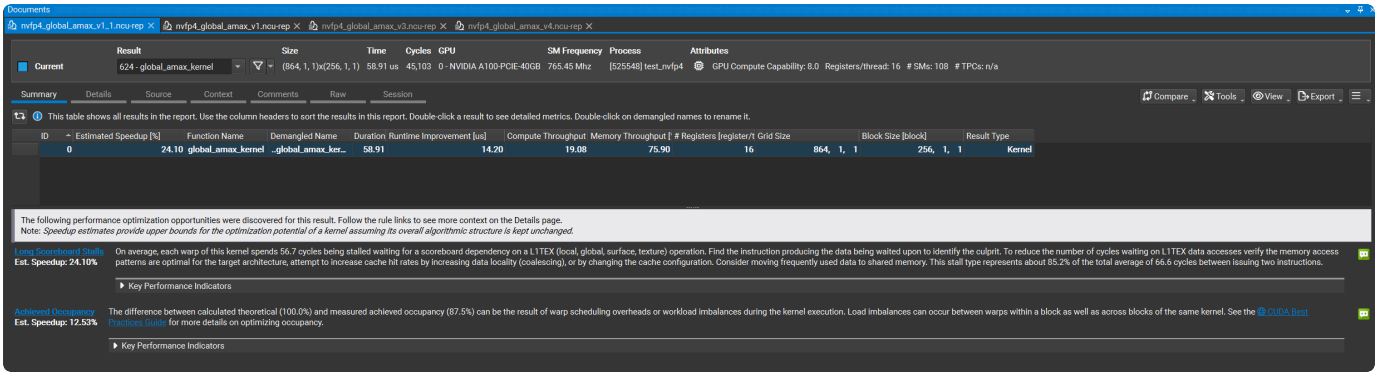
结果：

Metric	V1
Duration	**80.96 µs**
Grid	256
Registers	16

Compute Throughput	11.50%
Memory Throughput	55.22%
Achieved Occupancy	**27.5%**
Long Scoreboard	**81.7%**

4.4.2.2. V1： 256 → 864 blocks

采用 108 SM × 8 blocks/SM = 864， 修改 grid cap为 256 → 864 。



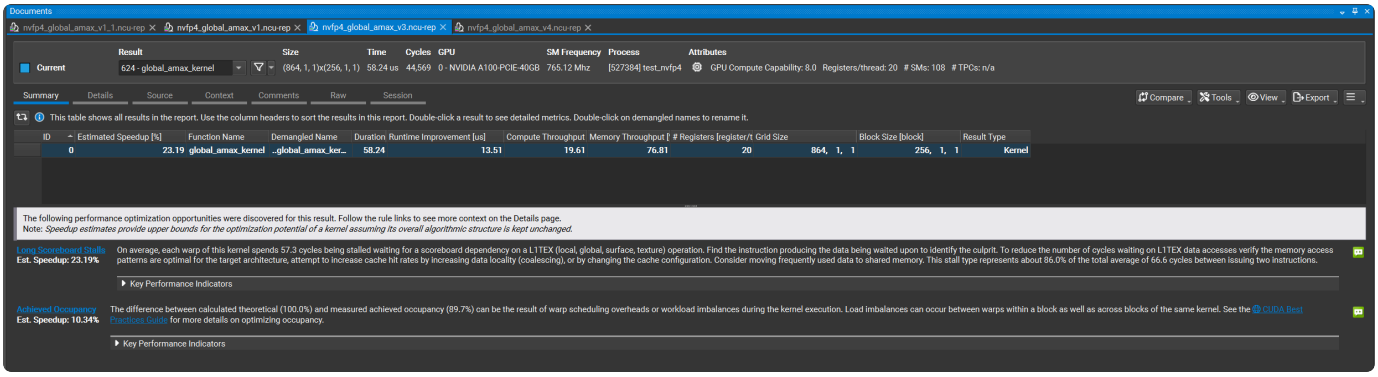
结果：

Metric	V1	V2
Duration	80.96	**58.91 µs**
Achieved Occupancy	27.5%	**87.5%**
Memory Throughput	55.22%	**75.90%**
Grid	256	864

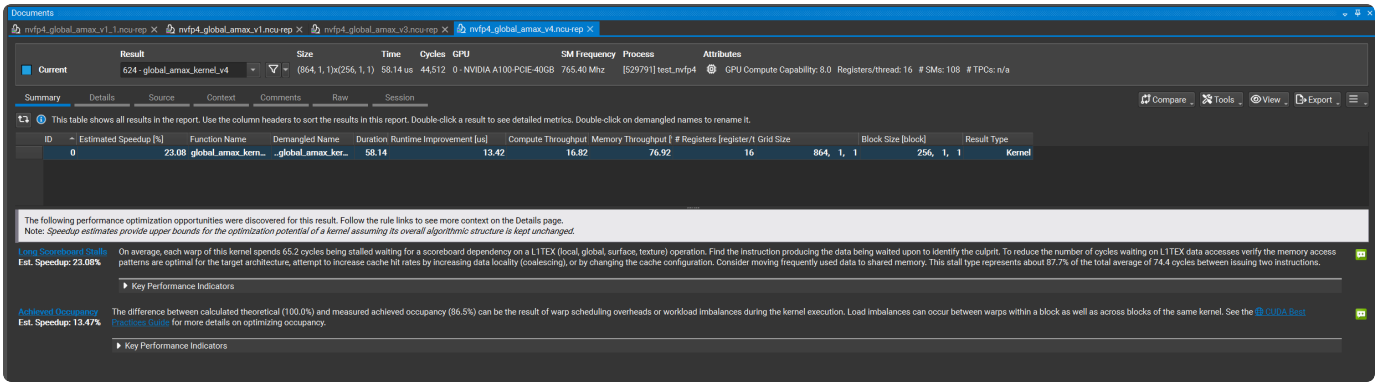
加速：1.37×

4.4.2.3. V2/V3 提升不大

V2：之后尝试对 global amax 使用四路独立 accumulator 增加 ILP，但只额外获得约 1% 收益，同时寄存器数量上升。



V3：将传统 shared-memory tree reduction 改为 warp shuffle reduction，每个 warp 先独立求最大值，只将八个 warp 结果写入 shared memory，再由 warp0 做最终 reduction。这减少了 shared-memory traffic 和同步次数，不过实际只进一步提升约 1.3%。因此 global amax 的主要收益来自提高 grid 并行度，而不是 reduction 本身。



4.4.3. NVFP4 Quantize Kernel

初始映射为：

1 thread → 2 values

8 threads

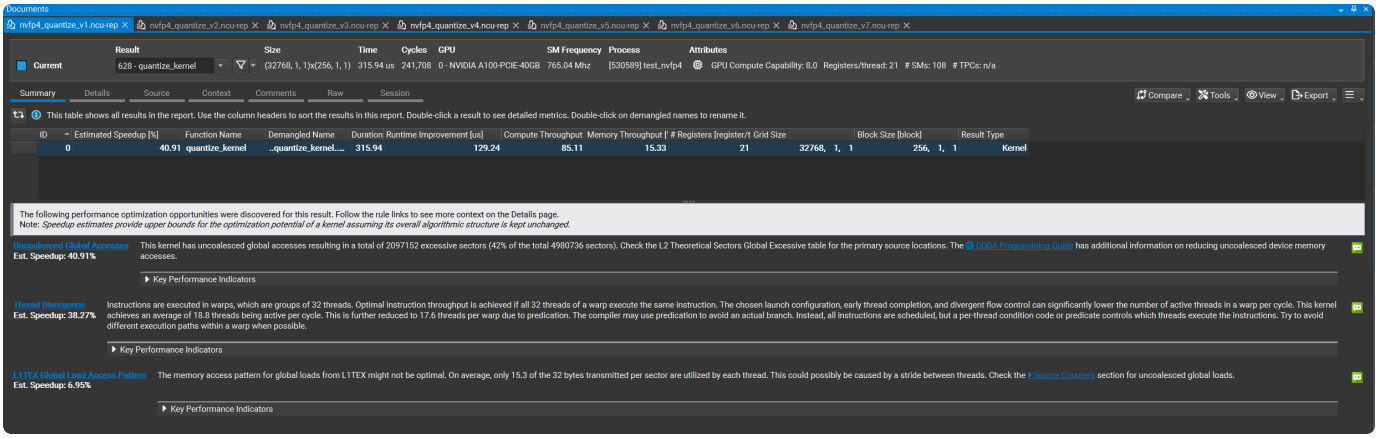
→ 16 values

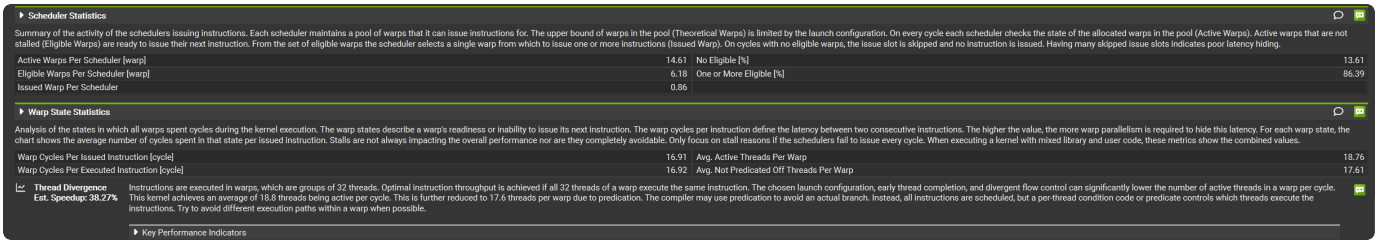
→ 1 NVFP4 block

1 warp

→ 4 NVFP4 blocks

4.4.3.1. NCU V1 Baseline





Metric	V1
Duration	**315.94 µs**
Registers	21
Compute Throughput	85.11%
Memory Throughput	15.33%
Avg Active Threads	18.8 / 32
Avg Predicated-On	17.6 / 32
Load useful bytes	15.3 / 32B

NCU 同时报告了严重的 uncoalesced load 和 thread divergence。

4.4.3.2. V2：Vectorized Input Load

原 FP32：

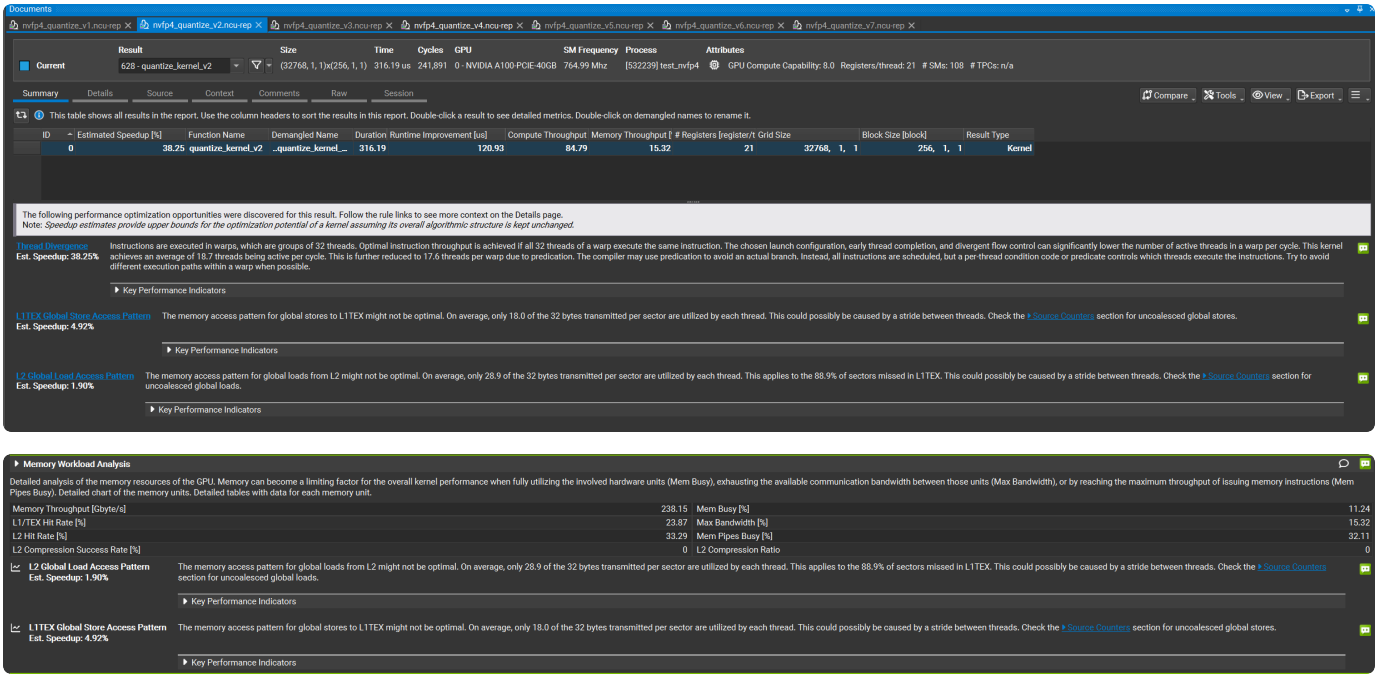
```
lane0 → input[0], input[1]
lane1 → input[2], input[3]
...
```

但两次 scalar load 分别形成 stride-2 access。

于是改成：

```
FP32 → float2
FP16 → ushort2
```

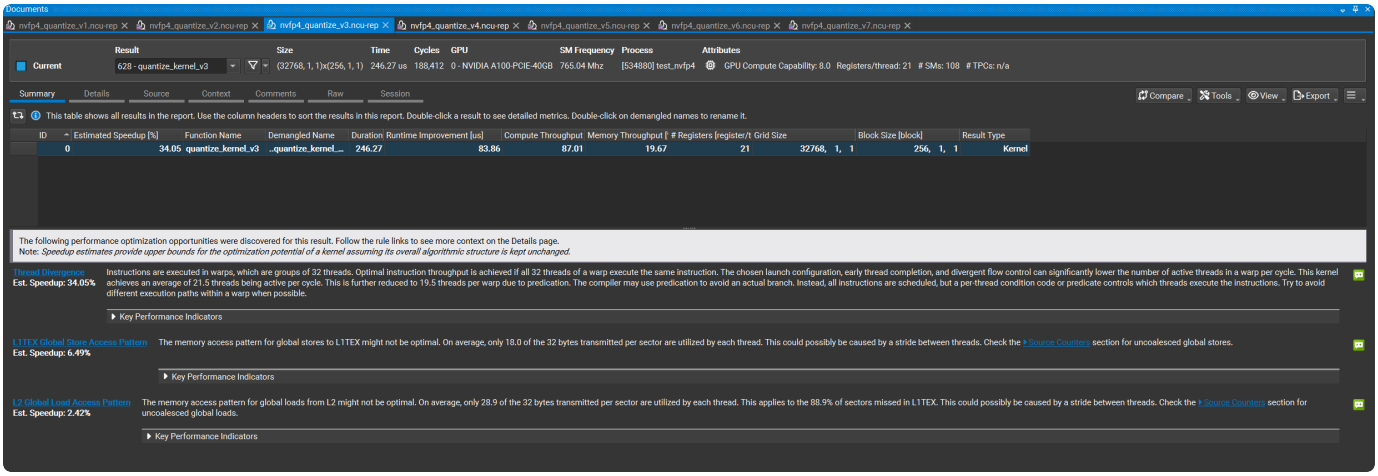
即使用 FP32 的 float2 和 FP16 的 ushort2 将两个标量 load 合并成 vector load。

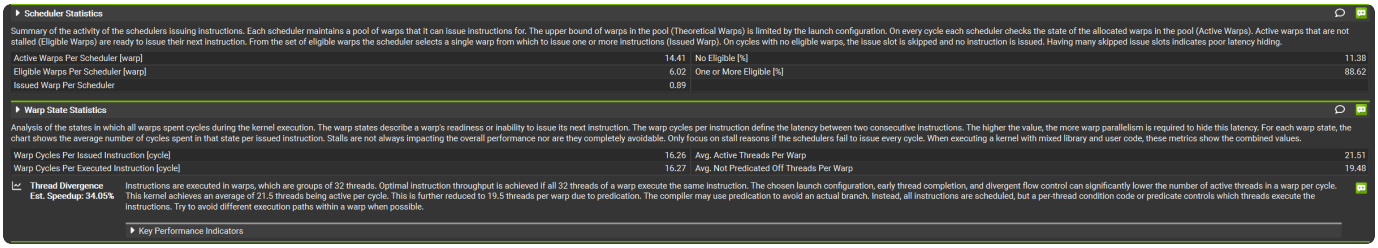


Load useful bytes从15.3 → 28.9 / 32B， 说明 memory transaction 修好了。 但 kernel 时间从 315.94 微秒变成 316.19 微秒，几乎完全没有改善。这说明 uncoalesced load 虽然是一个真实 warning，但并不是性能主因。

4.4.3.3. V3：Branch-Reduced E2M1 Encoding

原来的 float_to_e2m1 是一个通用浮点格式转换实现，需要处理 exponent、mantissa、normal、subnormal、round-to-nearest-even、carry 和 saturation，其中包含较多数据相关分支。而 E2M1 实际只有 0、0.5、1、1.5、2、3、4、6 这几个正数值，因此可以直接利用它们之间的 midpoint 做比较，并通过严格大于和大于等于的交替保留 ties-to-even 行为。





结果：

Metric	V2	V3
Duration	316.19	**246.27 µs**
Active Threads	18.73	**21.51**
Predicated-On	17.57	**19.48**
Registers	21	21

加速：1.28×

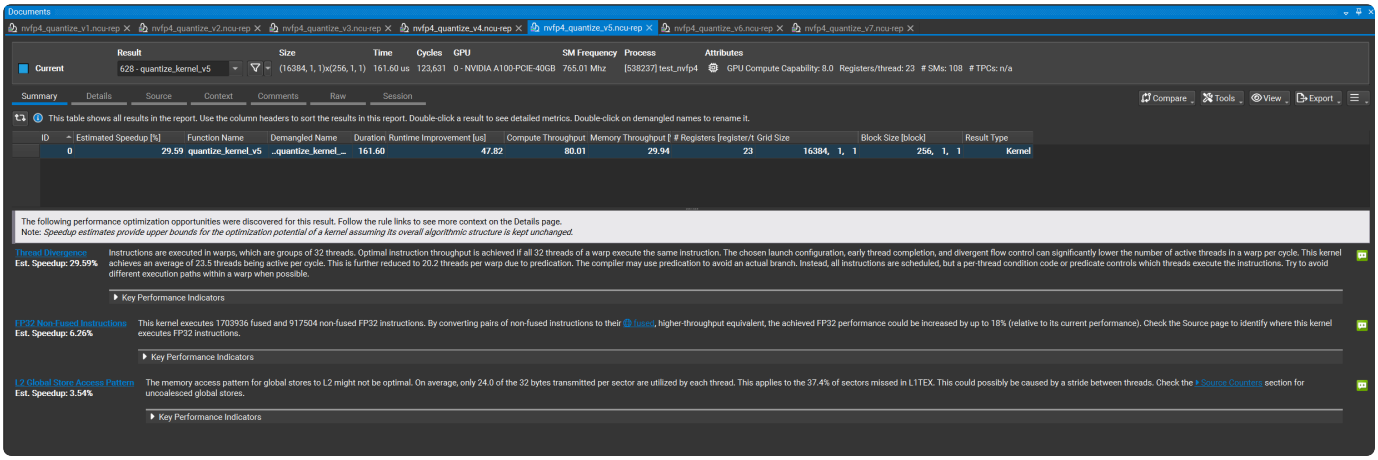
这个 branch-reduced 编码器将 Quantize 从 316.19 微秒降低到 246.27 微秒，同时 Active Threads 和 Predicated-On Threads 都明显增加。这证明 NVFP4 Quantize 的主要问题之一确实是计算路径和 SIMT utilization，而不是 memory bandwidth。

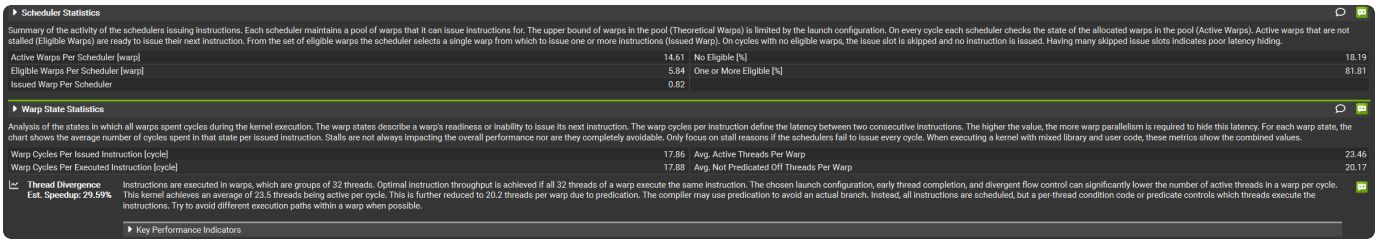
4.4.3.4. V4：共享倒数计算

原来每个线程的两个值都要分别除以 global scale 和 block scale，相当于一个线程有四次 FP32 division。由于同一个 16-value block 的 scale 是共享的，因此改成由 subgroup leader 只计算一次 `1 / (global_scale × block_scale)`，然后通过 shuffle 广播给 subgroup，其余 lane 直接使用乘法。这样把大量 division 替换成一个共享 倒数 和若干 multiplication。

4.4.3.5. V5：1 Thread → 4 Values

原来八个线程负责一个 NVFP4 block，因此每个 warp 只有四个 scale leader。计算 block scale 时只有 lane0、8、16、24 四个 lane 真正工作，也就是只有 12.5% 的 lane active。于是将映射改为每个线程处理四个值，只需要四个线程共同负责一个 16-value block，一个 warp 可以同时处理八个 block。这样 scale leader 数量由四个提高到八个，scale 计算阶段的活跃 lane 比例从 12.5% 提升到 25%。





结果：

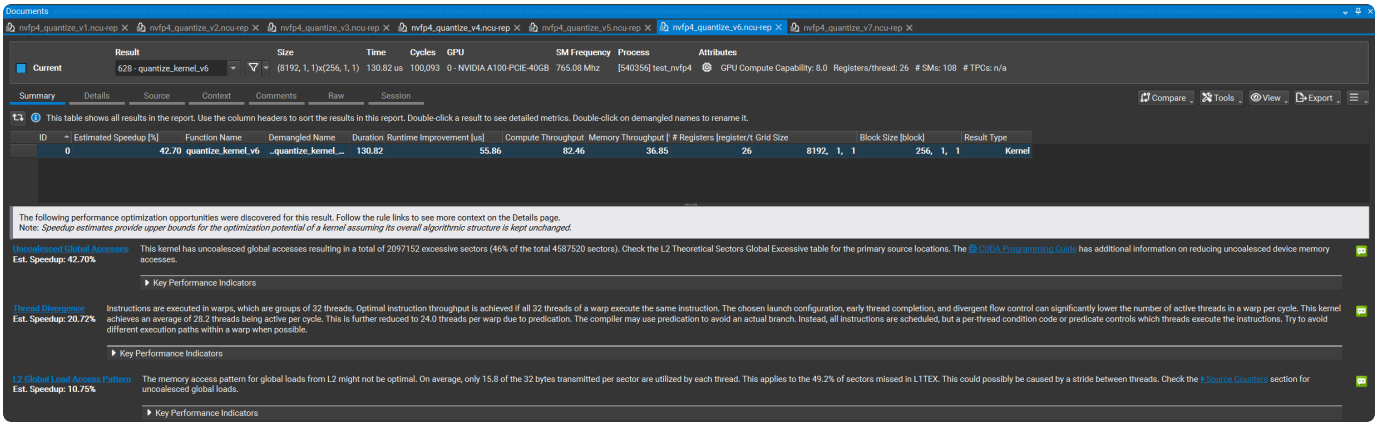
Metric	V4	V5
Duration	230.69	**161.60 μs**
Registers	19	23
Grid	32768	**16384**

Active Threads	18.7	**23.5**
Predicated–On	16.5	**20.2**

V4 → V5：1.43×

4.4.3.6. V6：1 Thread → 8 Values （继续线程粗化）

让每个线程处理八个值，只需要两个线程负责一个 NVFP4 block，因此一个 warp 可以同时处理十六个 block。scale leader 数增加到 16 个，也就是 scale 计算阶段可以有一半 warp lane 同时工作。





结果：

Metric	V5	V6
Duration	161.60	**130.82 µs**
Registers	23	26
Memory Throughput	29.94%	**36.85%**
Active Threads	23.46	**28.21**
Predicated–On	20.17	**23.96**

Active Warp	14.61	14.68
Eligible Warp	5.84	5.83

V5 → V6：1.235×

Active Threads Per Warp 最终提高到 28.21，Predicated-On Threads 也接近 24，而 Active Warp 和 Eligible Warp 并没有因为寄存器从 23 增加到 26 而明显下降。这说明还没有撞上明显的 register/TLP 墙。最终 kernel 时间进一步下降到 130.82 微秒。

4.4.3.7. NVFP4 Quantize 最终结果

Version	优化	Duration
V1	Baseline	315.94 μs
V2	vector load	316.19 μs
V3	branch-reduced E2M1	246.27 μs
V4	subgroup reciprocal	230.69 μs
V5	4 values/thread	161.60 μs
V6	8 values/thread	**130.82 μs**

最终：

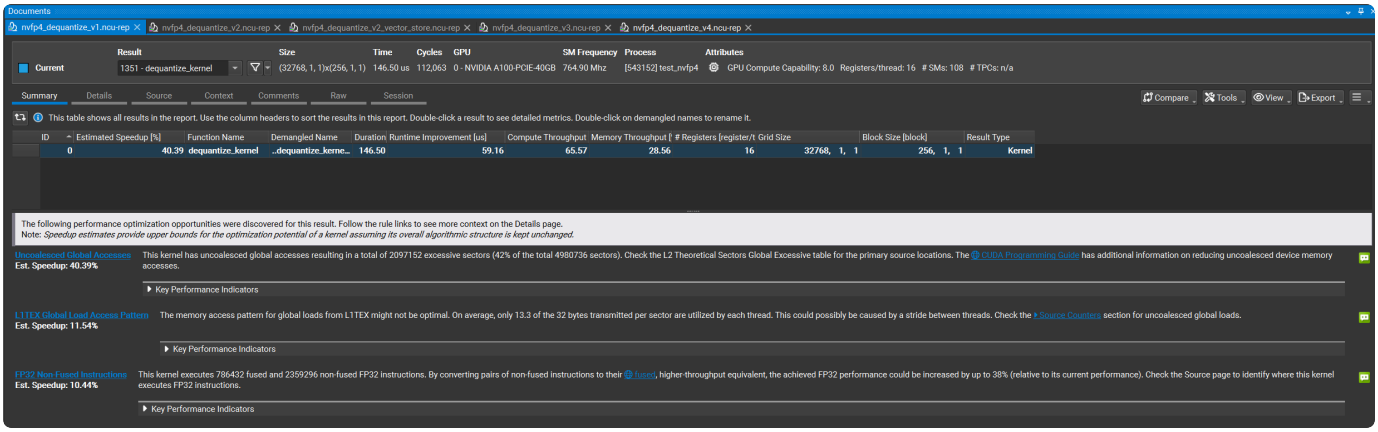
$$\boxed{\frac{315.94}{130.82} \approx 2.42\times}$$

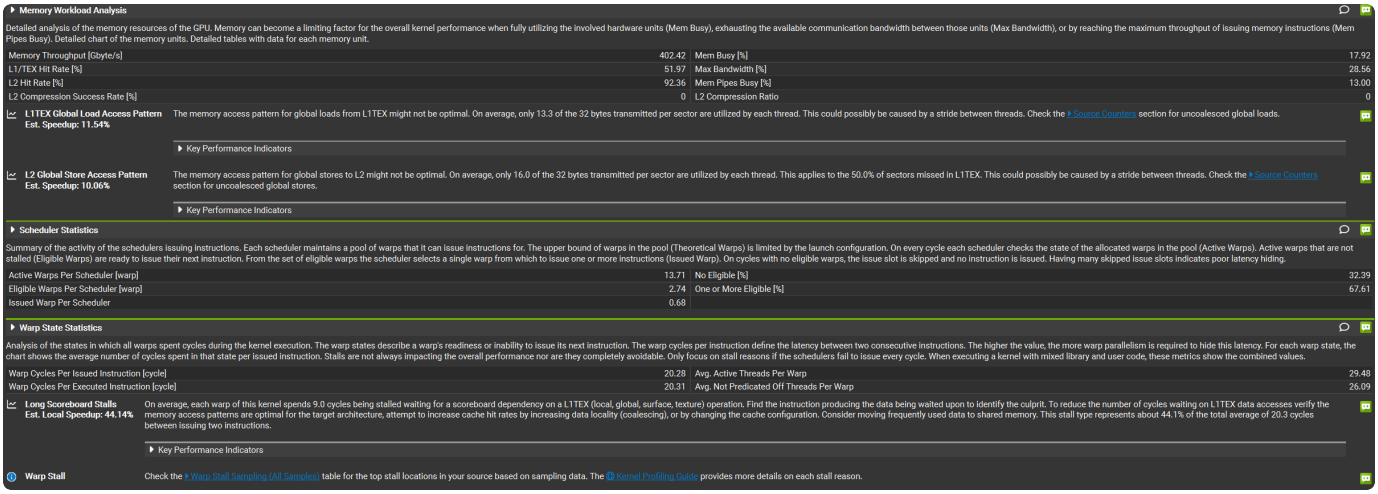
延迟下降：58.6%

4.4.4. NVFP4 Dequantize Kernel

baseline映射：**每个线程负责处理 1 个 packed byte，而 1 个 packed byte 中包含 2 个 E2M1 低精度数值。** 线程首先加载这个 packed input，同时读取该数据块对应的 block scale，以及整个张量共享的 global scale。随后，线程从这个 byte 中分别解码出两个 E2M1 数值，并结合 block scale 和 global scale 完成反量化，最终输出 2 个恢复后的数值。

4.4.4.1. V1





Metric	V1
Duration	**146.50 µs**
Registers	16
Compute Throughput	65.57%
Memory Throughput	28.56%
Load useful bytes	**13.3 / 32B**

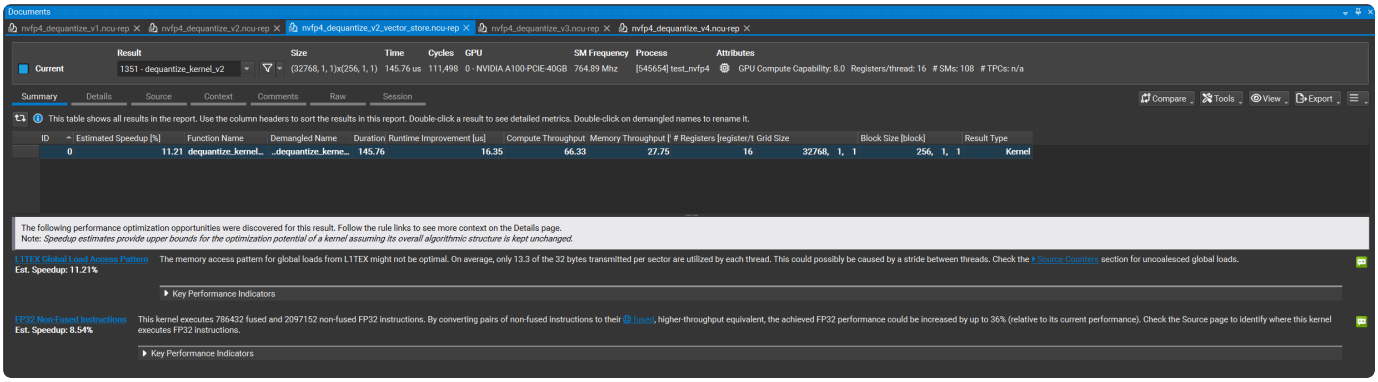
原始 Dequantize 中，每个线程会连续生成两个输出值 output[2*tid]和output[2*tid+1]， 如果编译器生成两条独立 scalar store，那么第一条 store 在一个 warp 内的地址类似：

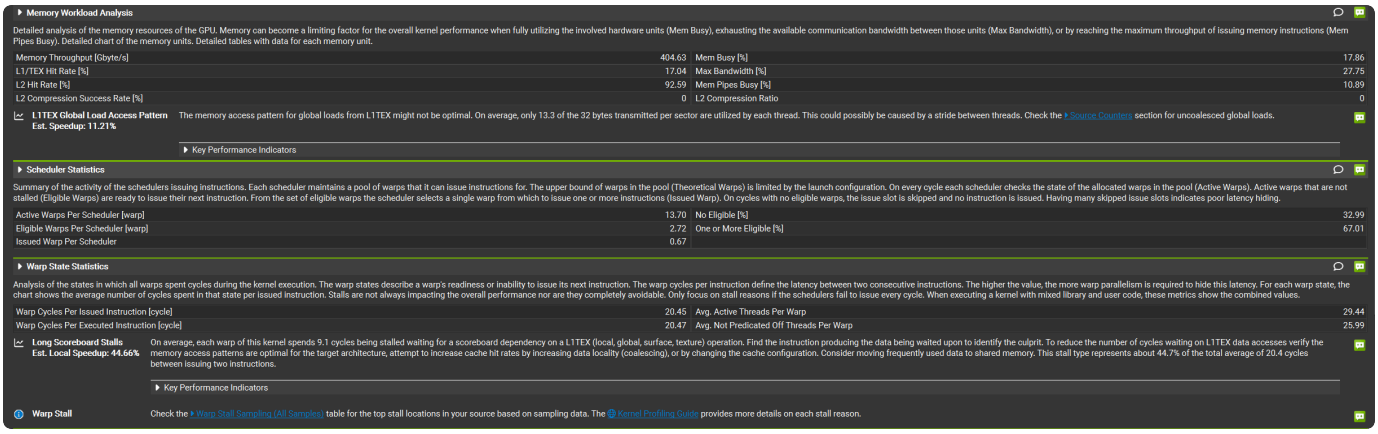
```
lane0 → output[0]
lane1 → output[2]
lane2 → output[4]
lane3 → output[6]
...
```

相邻 lane 之间间隔 8B，但每个 lane 实际只写 4B，因此单独看这一条 store，每个 32B sector 中大约只有一半字节真正有用。所以 NCU 会认为这种 store pattern 不够理想。

4.4.4.2. V2：Vectorized Output Store

尝试将两个 FP32 输出改成 float2 store，将 FP16/BF16 改成 ushort2 store。

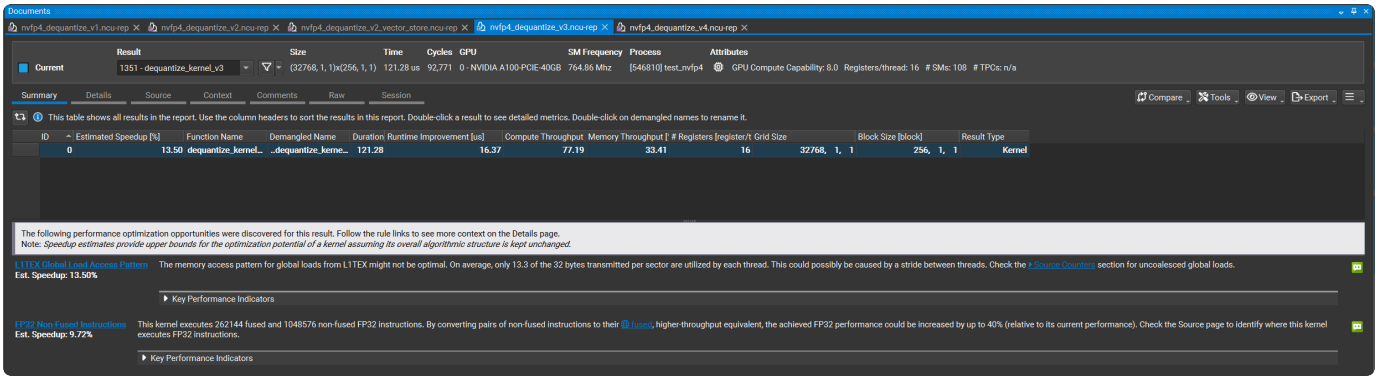


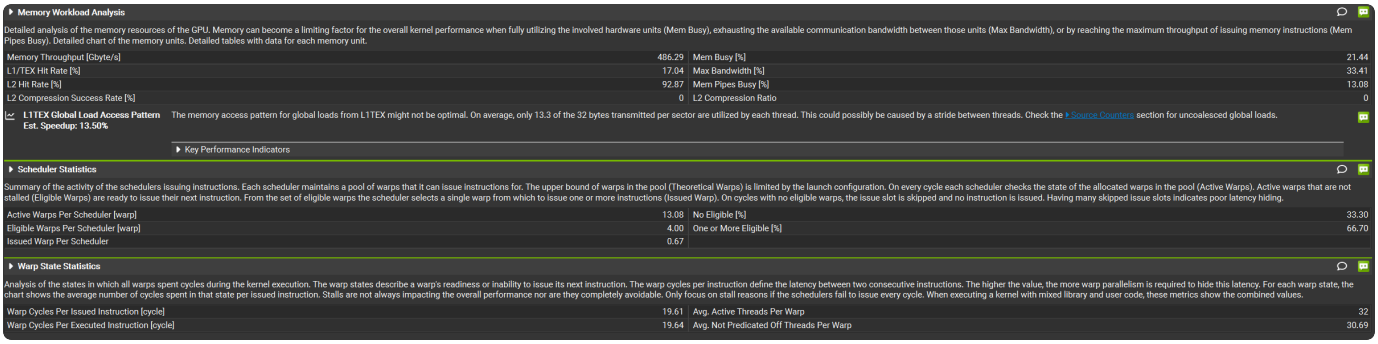


Global Store warning 被消除了，但 kernel 时间只从 146.50 微秒降低到 145.76 微秒，收益不到 1%，说明 store 不是主瓶颈。

4.4.4.3. V3：Bit-Level E2M1 Decode

E2M1 只有 16 种 bit pattern，因此没有必要通过通用的浮点算术和 ldexpf 来解码。将 sign、exponent 和 fraction 直接映射到 IEEE FP32 bit pattern 后，E2M1 decode 变成了主要由整数 shift、OR 和 __uint_as_float 组成的路径。这和之前编码时进行的优化是类似的。





结果：

Metric	V2	V3
Duration	145.76	**121.28 μs**
Compute Throughput	66.33%	**77.19%**
Memory Throughput	27.75%	**33.41%**
Registers	16	16

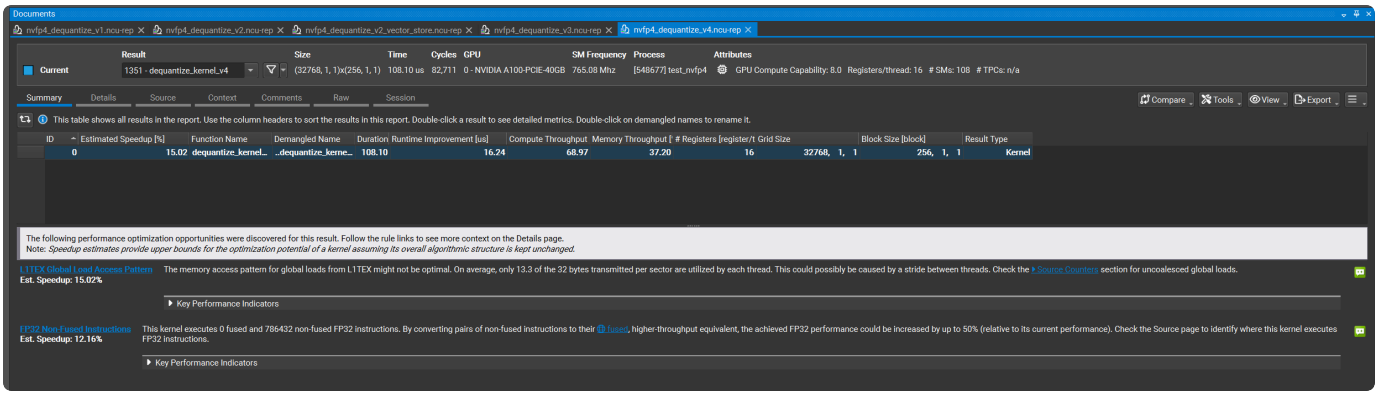
收益：16.8%

4.4.4.4. V4：Bit-Level E4M3 Scale Decode

对 E4M3 block scale 使用同样思路。由于 NVFP4 的 block scale 是自己生成的**非负 E4M3 scale**，可以针对 normal 和 subnormal 情况直接构造 FP32 exponent 与 mantissa，避免通用 e4m3_to_float 中的分支、浮点计算和 ldexpf。

```
normal:
FP32 exponent = E4M3 exponent + 120
FP32 mantissa = frac << 20

subnormal:
normalize 3-bit frac
→ 构造 FP32 bits
```





结果 121.28 → 108.10 μs 。

4.4.4.5. NVFP4 最终结果

Kernel	Baseline	Final	Speedup
Global Amax	80.96 μs	**58.14 μs**	**1.39×**
Quantize Kernel	315.94 μs	**130.82 μs**	**2.42×**
Dequantize Kernel	146.50 μs	**108.10 μs**	**1.36×**

5. 输出展示

```
------------------------------------------------------------
项目要求：三类矩阵端到端误差与性能测试
矩阵形状：4096 x 4096
warmup=10, benchmark iterations=100
============================================================

------------------------------------------------------------
数据分布：均匀随机矩阵
------------------------------------------------------------

[FP32 -> MXFP8 -> FP32]
元素数量            : 16777216
scale 数量          : 524288
有限值检查          : PASS
最大绝对误差        : 2.500000000e-01
MAE                 : 8.331946101e-02
MSE                 : 1.190223801e-02
压缩率              : 3.8788x
量化 kernel 时间    : 0.112876 ms
反量化 kernel 时间  : 0.095805 ms
量化有效带宽        : 747.818 GB/s
反量化有效带宽      : 881.060 GB/s

[FP32 -> MXFP8 -> FP16]
元素数量            : 16777216
scale 数量          : 524288
有限值检查          : PASS
最大绝对误差        : 2.500000000e-01
MAE                 : 8.331946101e-02
MSE                 : 1.190223801e-02
压缩率              : 3.8788x
量化 kernel 时间    : 0.112916 ms
反量化 kernel 时间  : 0.099154 ms
量化有效带宽        : 747.547 GB/s
反量化有效带宽      : 512.899 GB/s

[FP32 -> MXFP8 -> BF16]
元素数量            : 16777216
scale 数量          : 524288
有限值检查          : PASS
最大绝对误差        : 2.500000000e-01
MAE                 : 8.331946101e-02
MSE                 : 1.190223801e-02
压缩率              : 3.8788x
量化 kernel 时间    : 0.112886 ms
```

```
============================================================
项目要求：三类矩阵端到端误差与性能测试
矩阵形状：4096 x 4096
warmup=10, benchmark iterations=100
============================================================

------------------------------------------------------------
数据分布：均匀随机矩阵
------------------------------------------------------------
==PROF== Connected to process 548677 (/home/node/cudadev/low_precision-emulation/nvfp4/test_nvfp4)
==PROF== Profiling "dequantize_kernel_v4": 0%....50%....100% - 47 passes

[FP32 -> NVFP4 -> FP32]
元素数量            : 16777216
scale 数量          : 1048577
有限值检查          : PASS
最大绝对误差        : 1.333333015e+00
MAE                 : 3.741549447e-01
MSE                 : 2.401128155e-01
压缩率              : 7.1111x
量化 kernel 时间    : 0.154112 ms
反量化 kernel 时间  : 46.124176 ms
量化有效带宽        : 932.146 GB/s
反量化有效带宽      : 1.660 GB/s

[FP32 -> NVFP4 -> FP16]
元素数量            : 16777216
scale 数量          : 1048577
有限值检查          : PASS
最大绝对误差        : 1.334634304e+00
MAE                 : 3.742615847e-01
MSE                 : 2.402896526e-01
压缩率              : 7.1111x
量化 kernel 时间    : 0.153856 ms
反量化 kernel 时间  : 0.126915 ms
量化有效带宽        : 933.697 GB/s
反量化有效带宽      : 338.745 GB/s
```